# Seasonal and Weather-Based HSL City Bike Allocation

**Chenyu Wu, Jizhou Wang and Jussi Honkio**
Technical implementation, evaluation and reflection

## 1. Project motivation and scope

Our mini-project asks where city-bike operators should allocate more bicycles or review station capacity across months and quarters. The canvas proposed combining journey histories, weather, maps and regression in a planning website for operators and HSL planners, with multiple forecast horizons, uncertainty and an exportable review list. [1]

The implementation covers Helsinki and Espoo in April-October 2024 and 2025. It separates monthly station forecasting for allocation review from daily city-wide weather scenarios, with historical hourly profiles explaining commute dynamics. It supports planning and sensitivity analysis, while stopping short of real-time station-hotspot prediction or exact capacity prescriptions.

## Data collection and backend structure

The Python downloader retrieves 14 monthly origin-destination CSVs from HSL, station metadata from an official ArcGIS service and daily weather observations from the Finnish Meteorological Institute (FMI). Journey fields include origin and destination IDs, timestamps, distance and duration. Metadata supply names, municipality, dock capacity and WGS84 coordinates. Cached non-empty downloads can be reused, reducing repeated requests. [2]

The repository reports 5,102,233 raw journeys and 4,951,245 retained after cleaning, a 97.04% retention rate. The retained totals are 2,507,175 in 2024 and 2,444,070 in 2025. The main analytical table contains 6,395 station-month rows. The metadata catalogue has 457 stations, while the web export represents 454 mapped stations appearing in the journey analysis and seven unmapped IDs. These counts describe different populations and should not be treated as interchangeable. [2,3]

Collection, cleaning, notebooks, figure generation and web-data export are separated into modules. The main runner connects downloading and aggregation; notebooks contain exploratory analysis and model comparison. This division worked well because computational work happens before the browser loads the results, and intermediate tables can be inspected independently. It also makes the technical work reproducible without requiring the presentation interface.

## Architecture and station-map functionality

Python performs downloading, cleaning, aggregation, modelling and export. Derived JSON stores station histories, hourly curves, commute peaks, daily weather, station-month predictions and city-day scenario parameters. HTML, CSS and JavaScript provide a static GitHub Pages interface. Leaflet renders station maps over OpenStreetMap tiles, and Chart.js renders analytical charts. Views filter precomputed results; scenarios evaluate exported coefficients in the browser, without a prediction server. [6]

The first of five dashboard modules is Station Map & Rebalancing Triage. Users filter by year, month or selected quarter, municipality and metric; map colour and size communicate net flow or activity. Search, sortable rankings, station popups, linked table-to-map navigation, map reset and CSV export support station review. A separate audit retains seven unmapped IDs without invented coordinates. Q4 contains October only because the operating season ends that month.

# 2. Preprocessing and weather integration

## A consistent analytical unit

The cleaner standardizes column names, parses numeric fields and station IDs, drops missing required values, and applies plausibility filters: duration 10 seconds to 24 hours, distance at least 10 metres, average speed at most 10 metres per second, and return time no earlier than departure. These rules may also remove legitimate short trips. [2]

Station departures and returns are counted separately and outer-joined by monthly source file. Net flow equals returns minus departures. Total activity is their sum, so summing activity across the network counts both journey endpoints. Monthly returns follow the source-file cohort; intraday aggregation instead uses each event's own timestamp. Month-boundary attribution deserves an explicit audit before inventory interpretations.

The intraday pipeline separates weekdays and weekends and assigns departures and returns to their respective hours. Dividing by calendar days of each type avoids day-count bias. Commute Dynamics & Peaks provides selectable city, season/month and comparison mode, 24-hour curves, a top-10 tidal-swing chart, a morning-versus-evening quadrant plot for the top 120 stations, and a commuter-archetype table. Residential feeders, workplace centres, deficit sinks and accumulators describe directional patterns rather than observed journey purposes. [6]

## Missingness and quality checks

Unmapped IDs retain missing geographic metadata and are excluded from maps. Missing baseline counterparts are excluded from model comparison rather than assumed to have zero demand. An absent station-month is not necessarily a verified zero: the table starts from recorded events, not a historical operating-status panel. The inspected cleaner does not explicitly deduplicate journeys, and capacities are not historically versioned. Unknown municipality also becomes non-Helsinki in the monthly model's binary feature, suggesting an explicit unknown category as an improvement.

## Weather processing and scope

FMI daily mean, minimum and maximum temperature, precipitation and snow depth come from Kaisaniemi and Tapiola. The repository reports 428 seasonal daily observations per station with complete coverage for all five fields. UTC weather timestamps are converted to Europe/Helsinki. Negative precipitation codes are converted to zero; monthly summaries define rainy days at 1 mm or more. Tests check conventions, coverage, key uniqueness and join integrity. [2,3]

Weather & Ridership provides year, city, day-type and weather-variable filters; a scatterplot with descriptive OLS trendline; rainfall-intensity groups; and a dual-axis chronological view. Across both seasons it uses 856 matched city-days. For 2025, temperature-departure correlations are 0.737 in Helsinki and 0.665 in Espoo; precipitation correlations are -0.251 and -0.289. Seasonality, daylight and holidays confound causal interpretation. Weather is excluded from station-month forecasting, but included in the separate retrospective city-day experiment. Wind and archived meteorological forecasts remain outside the implementation. [3,6,7]

# 3. Forecasting design and evaluation

## A temporal benchmark

We compare a previous-year same-month baseline, Ridge regression and a decision tree. Departures and returns are modelled separately, and predicted net flow is their difference. The baseline assigns each 2025 station-month the same station's volume in the corresponding month of 2024. Across 3,170 matched April-October records, its departure MAE is 147.60 trips per station-month and WAPE is 19.23%; 27 unmatched records are excluded. [3] The Demand Forecasting Benchmarks tab exposes model, month, city, target and low-use filters, station search, observed-versus-predicted scatterplots, monthly WAPE, top-20 outflow overlap and a filtered net-flow roster. Its scorecard separates the full-season baseline from the common modelling cohort. [6]

## Correcting feature leakage

An earlier exploratory feature used each station's full-season mean, including the target month and later months in the training season. The notebook identifies this as leakage and replaces it with the preceding calendar month from the same season. At the start of month m, month m-1 is complete and available. Training uses May-October 2024; testing uses May-October 2025. April supplies the initial lag and is not forecast by the lag models. This correction showed why information availability at prediction time must govern feature design. [4]

Each regression has ten features: the relevant prior-month count, capacity, latitude, longitude, a Helsinki indicator and five month indicators. Missing numeric metadata use training-set medians. Ridge uses training-fitted standardization and alpha = 10. The tree uses maximum depth 6, minimum leaf size 10 and random seed 42. Predictions are clipped at zero. The comparison has 2,738 training observations and a shared cohort of 2,718 test observations requiring both a lag and a same-month baseline counterpart. [4,5]

| Model | Departure MAE | Departure WAPE | Return MAE | Return WAPE |
|---|---|---|---|---|
| Same-month baseline | 141.88 | 17.58% | 139.73 | 17.31% |
| Ridge regression | 226.01 | 28.00% | 226.52 | 28.06% |
| Decision tree | 199.53 | 24.72% | 195.63 | 24.23% |

Table 1. Shared May-October 2025 cohort, N = 2,718. MAE: completed trips per station-month. Source: repository model summary [5].

## What worked and what did not

MAE averages absolute count errors. WAPE is 100 times summed absolute errors divided by summed observed counts. The baseline has the lowest overall error on the common cohort. Matching the calendar month may preserve station scale and seasonality better than a recent-month lag, but this is an interpretation, not a demonstrated causal mechanism.

Ridge performs better on another objective: it captures an average 69.2% of the actual top-20 net-outflow stations, compared with 60.8% for the baseline across six months. Yet among the 681 low-use test observations with at most 198 departures, Ridge has 124.07% departure WAPE versus 33.83% for the baseline. This subgroup is defined retrospectively from test outcomes. Count accuracy, ranking quality and quieter-station performance must therefore be assessed separately. [4,5]

# 4. City-day weather scenario modelling

## A separate experiment with weather

The fifth module, City-Day Weather Scenario Explorer, asks how daily city-wide departures vary under hypothetical temperature and rainfall. It is separate from monthly station allocation. Two pooled Ridge models, each with alpha = 1 and an intercept, are fitted to 428 city-days in 2024 and evaluated on the same 428 city-days in 2025: 214 dates for each municipality. The calendar model uses a city indicator, weekend indicator and six month indicators, with April as reference. The weather model adds daily mean temperature and precipitation. The exporter fits these raw features without standardization and clips negative predictions at zero. [7]

Actual 2025 weather is used for retrospective conditional evaluation. This measures performance when weather is known; it is not a prospective forecast using weather available before the day begins. The two cities share regional conditions, so 428 city-days should not be interpreted as 428 independent dates.

| 2025 test cohort | Calendar WAPE | Weather WAPE | Weather MAE |
|---|---|---|---|
| Both cities (N=428) | 28.78% | 24.09% | 1,367.91 |
| Helsinki (N=214) | 21.38% | 17.86% | 1,810.63 |
| Espoo (N=214) | 90.54% | 76.13% | 925.20 |

Table 2. Calendar-only versus weather-augmented Ridge on identical 2025 city-days. MAE: completed departures per city-day. Stored results [7].

## Interactive implementation and error context

Users select municipality, month and weekday/weekend, then adjust temperature and rainfall sliders. JavaScript evaluates the exported coefficients and shows weather-augmented departures, the calendar-only estimate, their difference and city-specific evaluation metrics. This difference compares two separately fitted models; it is not an isolated causal weather effect. The fitted weather slopes are approximately +258 trips per degree Celsius and -71 trips per millimetre, shared across both cities under the additive specification. [7]

The interface adds each city's 10th and 90th percentiles of 2025 residuals to the scenario estimate, clipping the lower band at zero. This is a historical empirical error band, not a confidence interval or independently calibrated guarantee of future 80% coverage. The same evaluation season supplies both metrics and residual bands. A range warning flags individual weather inputs outside observed 2024-2025 bounds; unusual combinations within those bounds can still lack support.

Five historical analogue days provide factual context: the browser ranks same-city 2025 records using month/day-type penalties and scaled temperature/rainfall differences, then displays observations, predictions and residuals. This is explanatory retrieval rather than another trained forecasting model. [6,7]

## What the results teach us

Weather improves WAPE over the calendar-only comparison overall and within each city, but Espoo's 76.13% error remains large. Shared absolute weather and calendar slopes across very different city volumes are a plausible weakness to investigate, not a proven explanation. City interactions, separate models or count-based approaches should be validated on additional seasons. The explorer supports sensitivity analysis; it does not predict station-level shortages or incoming hourly hotspots.

# 5. Reflection, learning and next steps

## Changes from the original canvas

The canvas connected technical work to a planning decision. We achieved two-season collection, station-level aggregation, five dashboard modules, monthly forecasting benchmarks and a city-day weather scenario experiment. The main change was separating planning scales: monthly station counts for allocation review, historical hourly profiles for commute analysis, and daily city-wide weather sensitivity. This made the implementation manageable and the outputs interpretable. Hourly station forecasts, archived weather forecasts, wind features and an optimized allocation policy remain unfinished.

Correcting feature leakage was a key methodological improvement. The monthly baseline outperforming more complex models showed the value of a strong reference. Conversely, weather improved the city-day model relative to its calendar-only baseline, while high Espoo error exposed a limitation hidden by aggregate performance. We learned to distinguish count accuracy, ranking quality and performance on smaller groups. A trip is not a physical bicycle, monthly net flow is not inventory, and scenario sensitivity is not a causal intervention effect.

## What we would do differently

If starting again, we would define forecast horizons, decision times and evaluation cohorts before building the interface. A shared modelling module would reduce duplicated notebook/export logic. Explicit timestamp and timezone validation in the common cleaner, duplicate audits, checksums, historical station-status panels and versioned capacities would strengthen data quality. The static website is economical and easy to distribute, but exports can become stale and external tiles/libraries require connectivity. Existing weather, export, UI, consistency and scenario tests check important contracts; their presence alone is not evidence that a fresh end-to-end run passes. [2,6,7]

## Future development

Additional seasons should support rolling validation, hyperparameter selection and a final untouched test. City-specific effects and interactions deserve comparison against the pooled scenario model. Error bands should be calibrated on a separate period and evaluated for coverage, including by city and weather regime. Archived forecasts with issue timestamps would permit prospective daily/hourly weather evaluation.

Operational recommendations require occupancy, availability and redistribution records to assess unserved demand and empty/full-station risk. Monthly net flow can cancel intraday reversals; completed journeys cannot count unsuccessful hires. A capacity or redistribution policy should therefore consider service levels, costs and neighbourhood coverage, then be assessed in an operator pilot. Aggregate publication, transparent missingness and source attribution remain essential.

**Evidence and reproducibility.** Based on repository source and stored results at commit c36f7ff0d708538e9f85b4bb22f030d33e7c91d5, accessed 24 October 2026. Source, stored results and all five live dashboard tabs were reviewed. The full raw-data pipeline and test suite were not rerun. Operational pilot outcomes are not claimed.

**References.** All paths refer to github.com/luosuo890/HSL_City_Bike_Project.

[1] HSL_City_Bike_Project_Canvas.md. [2] README.md; src/download_data.py; src/clean_and_aggregate.py; src/aggregate_intraday.py; tests/test_weather_integration.py. [3] reports/findings.md. [4] notebooks/model_comparison.ipynb. [5] docs/data/data_model_summary.json. [6] docs/index.html; docs/js/map.js, commute.js, weather.js and forecast.js; src/export_web_data.py. [7] docs/data/data_weather_scenario.json; export_weather_scenario_data() in src/export_web_data.py; tests/test_weather_scenario.py.